

2.1 & 2.2 Introduction to CT & Algorithms

کمپیوٹیشنل تھنکنگ (Computational Thinking) مسائل حل کرنے کا ایک منظم طریقہ ہے۔

CS XI

Chapter 2: Computational Thinking & Algorithm

Lecture 01 of 11

★ Golden

2.1 & 2.2 Introduction to CT & Algorithms ★ Golden

LEARN IT

Computational Thinking (CT) is a systematic approach to solving problems.

"Systematic" means you follow a fixed, organised method instead of guessing. CT involves four activities:

- **Decomposition** — breaking a big problem down into smaller parts.
- **Pattern recognition** — identifying things that repeat or follow a rule.
- **Abstraction** — hiding (ignoring) details that do not matter.
- **Algorithms** — developing clear step-by-step instructions to solve the problem.

CT is not "thinking like a computer". It is thinking in a way that lets *you* describe a solution so clearly that a computer (or another person) could carry it out.

Problem → Decomposition → Pattern recognition → Abstraction → Algorithm → Program

You already use computational thinking in daily life. When you plan your exam preparation, you split the syllabus into chapters (decomposition), notice that every chapter needs the same routine of reading, making notes and practising (pattern recognition), ignore things that are not in the syllabus (abstraction), and make a day-by-day timetable (algorithm).

LEARN IT

Did you know? Pseudocode is a human-readable representation of an algorithm using plain English and structured programming keywords. It is the bridge between human logic and machine code. You will study it in detail in section 2.2.2.

REMEMBER IT

Remember the four parts of CT with the phrase "**Don't Pick Any Apple**": **D**ecomposition, **P**attern recognition, **A**bstraction, **A**lgorithm.

COMMON MISTAKE

Students sometimes think computational thinking means "writing code". It does not. CT happens *before* coding. You can do all four steps with pen and paper, without a computer.

اردو میں سمجھیں

کمپیوٹیشنل تھنکنگ (Computational Thinking) مسائل حل کرنے کا ایک منظم طریقہ ہے۔ اس کے چار حصے ہیں: مسئلے کو چھوٹے حصوں میں توڑنا (Decomposition)، دہرائی جانے والی باتوں کو پہچاننا (Pattern Recognition)، غیر ضروری تفصیلات کو چھپانا (Abstraction) اور مرحلہ وار حل لکھنا (Algorithm)۔ یہ سوچنے کا طریقہ ہے، کوڈ لکھنا نہیں۔

CHECK YOURSELF

1. Name the four parts of computational thinking.
2. Is computational thinking done before or after writing code?
3. What is pseudocode called a "bridge" between?

Answers: 1. Decomposition, pattern recognition, abstraction, algorithms. 2. Before coding (it is planning the solution). 3. Between human logic and machine code.

LEARN IT

An **algorithm** is a step-by-step logical procedure used to solve a problem. It is **conceptual** in nature (it is an idea, a plan) and completely **independent of any programming language**. The same algorithm can be written later in Python, C++ or Java.

A good algorithm has:

- **Input** — the data it starts with.
- **Clear steps** — each step has only one meaning.
- **A definite end** — it must finish after a limited number of steps.
- **Output** — the result it produces.

WORKED EXAMPLE

Algorithm to find the larger of two numbers

Step 1: Start

Step 2: Read A and B

Step 3: If $A > B$, print "A is larger"

Step 4: Else print "B is larger or equal"

Step 5: End

Notice: there is no Python or C++ here. Anyone — even a non-programmer — can follow these steps. That is what "language-independent" means.

2.2.1 Importance of Algorithms

Reason	What it means for you
Structured approach	Algorithms provide a structured approach to problem-solving, making complex tasks manageable.
Optimisation	They optimise time and memory usage, which is crucial for large-scale applications (for example, searching millions of records).
Reusability	They can be reused for similar problems, saving time and effort. A sorting algorithm for marks can also sort prices.
Accuracy and consistency	They ensure accuracy and consistency, reducing human error. The same steps always give the same result.
Blueprint	They act as a blueprint (a plan) before actual code implementation, just like an architect's map before building a house.

REMEMBER IT

Remember the five reasons as **S-O-R-A-B**: **S**tructured, **O**ptimised, **R**eusable, **A**ccurate, **B**lueprint.

COMMON MISTAKE

Do not write "an algorithm is a program". A program is the algorithm *written in a programming language* so that a computer can run it. The algorithm itself is only the logic.

الگورتھم (Algorithm) کسی مسئلے کو حل کرنے کا مرحلہ وار منطقی طریقہ ہے۔ یہ ایک تصور ہے اور کسی بھی پروگرامنگ زبان پر منحصر نہیں۔ الگورتھم کام کو منظم بناتے ہیں، وقت اور میموری بچاتے ہیں، دوبارہ استعمال ہو سکتے ہیں، غلطیاں کم کرتے ہیں اور کوڈ لکھنے سے پہلے نقشے کا کام دیتے ہیں۔

EXAM TIP

Very common short question: "Define algorithm and write its importance." Write the definition word for word — *a step-by-step logical procedure used to solve a problem; conceptual and independent of any programming language* — then list at least four of the five importance points. If asked to "write an algorithm", always begin with **Step 1: Start** and finish with **End**, and number every step.

CHECK YOURSELF

1. Why is an algorithm called "language-independent"?
2. Give two reasons why algorithms are important.
3. Write a 5-step algorithm to find the sum of two numbers.

Answers: 1. Because it describes only the logic; it is not tied to Python, C++ or any other language. 2. Any two: structured approach, optimise time/memory, reusable, accurate and consistent, blueprint before coding. 3. Step 1: Start; Step 2: Read A and B; Step 3: SUM = A + B; Step 4: Print SUM; Step 5: End.

Extra board practice

These MCQs and questions are extra to the textbook. Close the notes, attempt them, then open the key.

MCQ 1. Computational thinking is:

- a. A systematic way to solve problems
- b. Only typing Python
- c. Only drawing gates
- d. A database key

MCQ 2. An algorithm is:

- a. A step-by-step finite method to solve a problem
- b. A random guess
- c. A picture of a PC
- d. An IP address

MCQ 3. CT pillars include:

- a. Decomposition, pattern, abstraction, algorithm
- b. Only encryption
- c. Only HTML
- d. Only Agile sprints

MCQ 4. Algorithms must halt:

- a. After finitely many steps
- b. Never
- c. Only on Fridays
- d. After OSI layer 1

MCQ 5. Cooking a recipe is like:

- a. An algorithm
- b. A K-map group of 3
- c. A XOR of pies
- d. Big O of clouds

MCQ 6. CT is useful even:

- a. Without a computer
- b. Only inside NAND
- c. Only in Figma
- d. Only in SQLite

Extra short questions

1. Define computational thinking.
2. Define algorithm with two properties.
3. Give a non-computer algorithm.
4. How is CT different from "just coding"?

Extra long question

1. Take "assign 200 students to exam rooms". Apply the four CT pillars and then write a 6-step algorithm.

Extra answer key

MCQs: 1. A 2. A 3. A 4. A 5. A 6. A

Short

1. Solving problems by breaking them down, spotting patterns, hiding detail (abstraction) and writing algorithms.
2. Ordered, finite, unambiguous steps that produce a result. Precision + finiteness.
3. Making tea / finding a roll number in a sorted list.
4. CT is the thinking; coding is one way to write the algorithm down.

Long

1. Decompose: rooms, students, constraints. Pattern: groups of 30. Abstract: ignore bag colours. Algorithm: numbered steps with a stop condition. Mention input/output.

Next lecture: 2.2.2 Algorithm vs Pseudocode